Emergent State Machines: A Deterministic Architecture for Situational Reasoning in Complex Systems

Richard Hines
Independent Researcher
Tacoma, Washington, USA
richardhinestacoma@gmail.com

Abstract

This paper presents the mathematical foundations of the Emergent State Machine (ESM), a deterministic architecture for interpretable situational reasoning in complex systems. The central organizing unit of the architecture is the *turn*: a complete reasoning frame in which observations from the environment are transformed into signals, assembled into an interpretable state vector, projected into policy-relevant coordinates, evaluated for relevance, and, when warranted, mapped deterministically to an action or other outcome. By structuring system behavior as a sequence of turns, the ESM makes reasoning processes inspectable, replayable, and auditable.

Within each turn, raw observations are converted into direct and derived signals defined by domain experts. These signals form an explicit state vector representing the system's current situation. A projection mechanism then expresses that state in policy-relevant coordinates, enabling deterministic policy selection without collapsing reasoning into latent or opaque representations. Repeated turns generate trajectories through interpretable and policy-relevant state space over variable real-world time, allowing policies to guide the system toward desirable stable regions and away from undesirable ones.

Unlike architectures that entangle probabilistic interpretation with consequential action, the ESM preserves a structural boundary between state representation and authoritative policy selection. Projection interprets; relevance gating determines whether a situation warrants decision; deterministic policy authorizes action. This separation supports traceability, failure localization, and controlled system evolution through versioned revision of signals, projections, gating logic, and policies. The resulting framework provides a mathematically structured approach to decision systems whose reasoning must remain visible, reproducible, and governable across domains including clinical monitoring, learning systems, and operational control.

github.com/emergent-state-machine/esm-spec

Contribution Statement. This paper makes five principal contributions. First, it defines the *turn* as the atomic reasoning object of an Emergent State Machine, providing a formal unit for inspection, replay, and audit. Second, it presents a deterministic state-space formulation in which observations are transformed into signals, state vectors, policy projections, relevance-gating states, and turn outcomes through an explicit reasoning chain. Third, it shows how repeated turns generate trajectories through projected state space over variable real-world time, enabling policy to guide the system toward desirable stable regions through persistence-aware intervention rather than instantaneous reaction alone. Fourth, it formalizes layered ESM reasoning in which initialized state may be transferred across interacting ESMs without collapsing modular authority. Fifth, it formalizes *Instrumented Deterministic Evolution* as a regime in which behavioral change occurs only through explicit revision of versioned architectural artifacts rather than through hidden parameter drift.

1 Introduction

Many contemporary decision systems operate as opaque pipelines in which complex transformations produce outputs without exposing the intermediate reasoning steps that led to those outputs. In domains such as clinical care, education, and operational control, this opacity creates a practical problem: professionals remain responsible for consequential decisions, yet the computational systems supporting those decisions often do not provide an inspectable account of how those decisions were reached. Threshold-triggered alerting systems partially address this problem by surfacing selected measurements, but they typically expose signals rather than situations. Human operators must still reconstruct the situational meaning of those signals for themselves.

The Emergent State Machine (ESM) addresses this limitation by structuring automated reasoning into discrete, inspectable units called *turns*. A turn is the fundamental reasoning frame of the system. During a turn, observations are collected from the environment, transformed into direct and derived signals, assembled into an interpretable state vector, projected into policy-relevant coordinates, evaluated through relevance gating, and, when warranted, mapped deterministically to an action or other outcome. The turn therefore serves as the minimal object required to inspect, replay, reproduce, and audit system reasoning.

This framing is important for both theory and practice. Theoretically, it allows the ESM to be modeled as a dynamical system evolving through interpretable and policy-relevant state spaces. Practically, it gives system designers and domain experts a concrete artifact that can be logged, examined, and revised. Rather than treating reasoning as an invisible process inside a model, the ESM makes each reasoning step explicit. Each turn packages the information available to the system, the transformations applied to that information, the policy dimensions activated, the relevance conditions satisfied, and the outcome produced. A sequence of turns

therefore provides both a trajectory through state space and a structured record of how the system's reasoning unfolded.

The architecture developed here differs from systems that entangle probabilistic interpretation with authoritative action selection. In the ESM, observations, signals, state representation, policy projection, relevance gating, and policy action remain structurally separated. Projection expresses what the current situation means in policy-relevant terms. Relevance gating determines whether the current situation warrants authoritative decision at all. Policy then governs which action, if any, is authorized. This separation preserves interpretability and makes system behavior traceable. The resulting reasoning chain can be inspected in both directions: from observations forward to outcome, and from outcome backward to the observations and rules that produced it. Because the architecture is deterministic at the authoritative layer, system behavior is replayable under fixed versions of its governing artifacts.

Time is explicit in this framework in two distinct senses. The sequence of turns provides a discrete, inspectable reasoning order, while real-world time between turns may vary depending on when observations arrive or when conditions warrant reevaluation. This distinction allows the ESM to represent persistence, abrupt change, and temporally decaying relevance without collapsing reasoning into a continuous opaque process.

This paper presents the mathematical foundations of that architecture. It first defines the turn as the atomic unit of reasoning, then formalizes the transformation from observations to signals, state vectors, projected policy coordinates, relevance-gating states, and outcomes. It next describes state evolution, temporal dynamics, layered initialization and handoff, trajectories, desirable regions, and stable states in projected state space. Finally, it shows how determinism, auditability, and replayability support a regime of *Instrumented Deterministic Evolution*, in which system behavior changes only through explicit revision of versioned architectural components.

- observations, representing measurements or events in the monitored environment
- signals, which are direct or derived quantities computed from observations
- state vectors, organizing signals into a structured representation of the current situation
- projection into policy space, evaluating the state relative to defined decision dimensions
- relevance gating, determining whether the current situation warrants policy evaluation
- policy selection and outcome determination, governing authoritative system response
- state evolution, occurring as the system receives new observations over time
- temporal dynamics, including persistence, volatility, abrupt change, and decay of relevance

By making each stage explicit and inspectable, the ESM provides a transparent alternative to black-box decision systems. Each outcome can be traced through a deterministic chain linking actions to rules, signals, and raw observations.

The architectural principles underlying the Emergent State Machine draw on ideas from control theory, dynamical systems, software architecture, and research on situation awareness and AI governance. The relationship between these traditions and the ESM architecture is discussed in Section 14.

2 The Turn as the Fundamental Unit of Reasoning

The Emergent State Machine organizes reasoning into discrete computational units called *turns*. A turn represents one complete cycle of observation, interpretation, relevance evaluation, and, when warranted, policy selection. Rather than treating system reasoning as a continuous and opaque transformation, the ESM treats each turn as an explicit object that can be inspected, replayed, and reproduced.

At time t, a turn may be represented abstractly as

$$T_t = (o_t, S_t, x_t, y_t, g_t, r_t) \tag{1}$$

where

- o_t denotes the observations available at time t,
- S_t denotes the set of direct and derived signals computed from those observations,
- x_t denotes the interpretable state vector constructed from those signals,
- y_t denotes the representation of the state in policy-relevant coordinates,
- g_t denotes the relevance-gating state,
- r_t denotes the turn outcome.

When an action is authorized, the action itself is denoted separately by a_t and is one possible component of the turn outcome.

This formulation makes explicit that a turn is not merely a timestamped input or output. It is the full reasoning frame of the system at that moment. Each turn contains the information needed to reconstruct the system's situational interpretation, the relevance conditions applied to that interpretation, and the policy logic that produced its outcome.

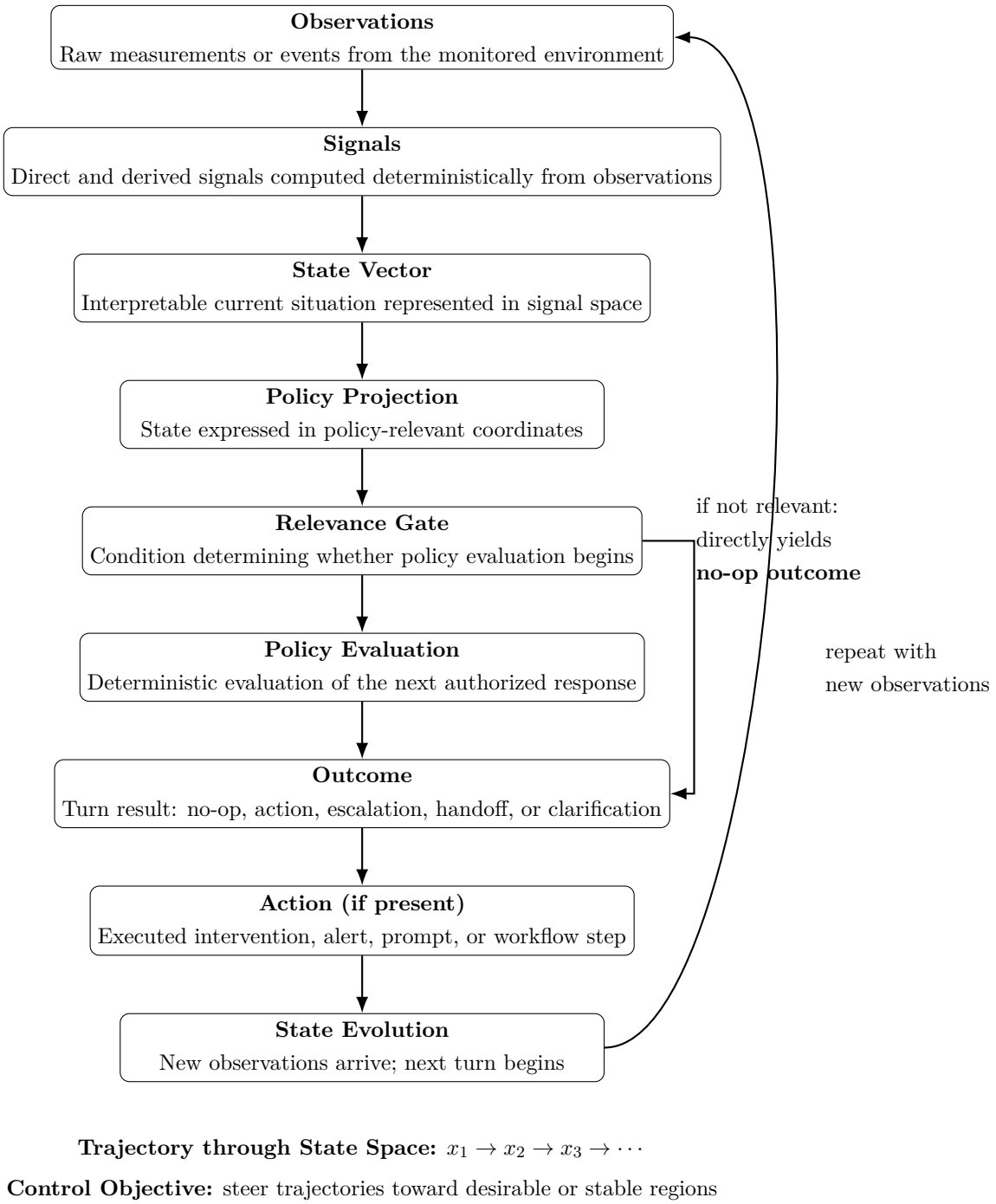

Figure 1: Core reasoning loop of an Emergent State Machine. Each turn transforms observations into signals, constructs a state vector, projects the state into policy coordinates, evaluates relevance, and, when warranted, selects an action or other outcome.

In practical deployments, the first operational turn need not begin from an empty case state. A turn may begin with initialized contextual information provided by prior system

structure, a distinct intake process, or an upstream ESM. What is required is not emptiness but explicitness: the initialized conditions, the observations available, and the versioned reasoning artifacts governing the turn must all remain inspectable.

A sequence of turns forms the temporal structure of ESM reasoning:

$$T_1 \to T_2 \to T_3 \to \dots \tag{2}$$

This sequence corresponds to the evolution of the monitored system through state space. Because each turn remains explicit and inspectable, the system's reasoning process can be analyzed at the level of individual turns or at the level of trajectories formed across many turns.

The concept of the turn plays two roles in the architecture. First, it is the atomic object of inspection and audit. Second, it is the temporal unit through which state trajectories are generated. In this sense, turns connect the operational behavior of the system to its mathematical formulation. What evolves over time is not an abstract hidden process, but a sequence of explicit turns whose contents remain available for replay and analysis.

3 Observations and Signals

Each turn of an Emergent State Machine begins with observations obtained from the environment being monitored. Observations represent measurements, events, or reported information describing the system at time t.

Observations are denoted

$$o_t \tag{3}$$

where o_t represents the collection of raw inputs available to the system during turn t. These inputs may originate from sensors, instruments, logs, or structured human reporting.

Observations themselves are not directly used for policy evaluation. Instead, they are transformed into *signals*, which provide structured, policy-relevant quantities derived from the available observations.

Signals fall into two categories: *direct signals* and *derived signals*.

3.1 Direct Signals

Direct signals correspond directly to measured or reported quantities within the observations. A direct signal is defined as

$$s_i^{direct} = m_i(o_t) \tag{4}$$

where m_i is a deterministic measurement function extracting the relevant value from the observations.

Examples of direct signals include quantities such as heart rate, respiratory rate, oxygen saturation, or a recorded student response in an educational setting.

Direct signals represent the immediate measurable properties of the system at the current turn.

3.2 Derived Signals

Derived signals represent deterministic computations defined over observations or other signals. These signals capture relationships, trends, persistence conditions, rates of change, or structural properties that are not directly observable but can be computed from available measurements.

A derived signal may be written as

$$s_j^{derived} = f_j(o_{t-k}, \dots, o_t) \tag{5}$$

or equivalently

$$s_j^{derived} = f_j(s_1, s_2, \dots) \tag{6}$$

where f_j is a deterministic function defined by system designers or domain experts.

Derived signals allow the system to represent domain-specific quantities such as ratios, differences, persistence conditions, temporally windowed volatility, or rates of change. For example, a common clinical derived signal is the shock index

$$\text{shock_index} = \frac{HR}{SBP}. \tag{7}$$

Derived signals may therefore incorporate temporal structure by referencing observations across multiple turns.

3.3 Signal Set

The complete set of signals available during turn t is denoted

$$S_t = \{s_1, s_2, \dots, s_n\}. \tag{8}$$

Signals provide the structured measurement interface between raw observations and the system's internal representation of the current situation. Because signals are explicitly defined and versioned, they remain interpretable and auditable across system revisions.

4 State Vector Construction

Signals derived during a turn are organized into a structured mathematical representation of the system's current condition called the state vector. Representing system behavior through explicit state vectors is standard practice in control and system identification theory [Ljung, 1999].

The practice of representing system conditions through structured feature measurements derived from observable inputs is common in statistical learning and signal-processing systems [Bishop, 2006, Raghavan and Kulkarni, 2014]. In the ESM architecture, these signals remain explicitly defined and inspectable rather than embedded within latent model parameters.

The state vector at time t is denoted

$$x_t \tag{9}$$

and constructed from the available signals as

$$x_t = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}. \tag{10}$$

Each component of the vector corresponds to a signal computed from the observations available during the current turn.

Unlike latent representations commonly used in many machine learning systems, the ESM state vector is explicit and interpretable. Each element corresponds to a signal whose definition, range, and computation are known. This design ensures that the system's representation of the current situation can be inspected, verified, and revised by domain experts.

The state vector therefore functions as the system's formal description of the situation being monitored at turn t. It captures the relevant measurable and derived properties of the system in a structured mathematical form suitable for projection and policy evaluation.

The state vector may include both instantaneous signals and temporally derived signals, provided that all components remain explicitly defined and reproducible. In layered deployments, the state vector may also incorporate initialized contextual information inherited from prior turns or from upstream system components, so long as that inherited structure remains inspectable.

Because the state vector is constructed deterministically from signals, identical observations and initialized conditions will always produce identical state vectors under fixed signal definitions. This property supports replayability and traceability of system reasoning.

Across successive turns, the state vector evolves as new observations arrive and new signals

are computed. The resulting sequence

$$x_1, x_2, x_3, \ldots \tag{11}$$

forms a trajectory through the system's state space, representing the evolution of the monitored situation over time.

5 Relevance Gating

Not every turn warrants authoritative decision. The Emergent State Machine therefore employs *relevance gating* to determine whether the current projected situation requires policy evaluation.

A relevance gate evaluates conditions defined over the projected state:

$$g_t = G(y_t) \tag{12}$$

where G is a deterministic gating function and

$$g_t \in \{0, 1\}. \tag{13}$$

If $g_t = 1$, the situation is treated as decision-relevant and policy evaluation proceeds. If $g_t = 0$, the system does not invoke policy and the turn outcome is a no-op.

Relevance gating serves three primary purposes.

First, it prevents the system from treating every turn as equally consequential. The ESM evaluates every turn, but it does not authorize policy evaluation on every turn. This distinction preserves a separation between continuous observation and conditional intervention, consistent with event-triggered control formulations in which control action is invoked only when relevance conditions are satisfied [Tabuada, 2007].

Second, relevance gating restricts authoritative decision to situations that satisfy explicit structural conditions. These conditions may include threshold crossings, persistence requirements, transitions into or out of desirable regions, abrupt change, elevated volatility, or other deterministic criteria defined by system designers and domain experts. In this sense, relevance gating functions as a supervisory layer over decision activation, analogous to supervisory control of discrete event processes [Ramadge and Wonham, 1987].

Third, relevance gating supports modular reasoning architectures. Different policy regimes or escalation pathways may become available under different gating conditions, but the activation of those regimes remains governed by explicit, inspectable functions of projected state. Because relevance gates are explicitly defined, the conditions under which policy becomes active remain inspectable and auditable. When a particular turn becomes decision-relevant, the gating condition responsible for that activation can be traced directly to the signals and projected dimensions that produced it.

Because relevance gates are explicitly defined, the conditions under which policy becomes active remain inspectable and auditable. When a particular turn becomes decision-relevant, the gating condition responsible for that activation can be traced directly to the signals and projected dimensions that produced it.

6 Projection into Policy Space

The state vector describes the current situation of the monitored system in terms of measurable signals. However, policy decisions are typically defined in terms of higher-level situational dimensions rather than individual signals. The Emergent State Machine therefore transforms the state vector into a representation expressed in policy-relevant coordinates.

This transformation is called *projection into policy space*.

Let P denote the policy projection operator. The projection of the state vector is

$$y_t = Px_t \tag{14}$$

where

- x_t is the state vector constructed from signals,
- P is the policy projection operator,
- y_t is the representation of the state in policy-relevant coordinates.

Projection is a required step of the architecture. Policy does not operate directly on raw observations or on unprojected state vectors. Instead, policy operates on a representation of state that has already been expressed in decision-relevant coordinates.

Each component of y_t corresponds to the activation level of a policy-relevant dimension. These dimensions are defined by system designers or domain experts and represent structural properties of the situation being monitored.

For example, in a clinical monitoring system, projection dimensions might include degrees of respiratory instability, circulatory compromise, persistence of improvement, or confidence in current interpretation. In an educational system, projection dimensions might represent claim strength, evidence relevance, reasoning coherence, recent volatility, or confidence in a current assessment.

Projection therefore does not predict the future. Instead, it illuminates the current situation by expressing the measured state in coordinates relevant to relevance gating and policy evaluation. Representing system behavior through transformations of state vectors into decision-relevant coordinates is consistent with state-space approaches in control and estimation theory [Kalman, 1960, Ljung, 1999].

In simple systems, projection may be close to an identity transformation. In more complex systems, projection may involve richer deterministic transforms or reproducible model-assisted mappings. In either case, projection remains structurally distinct from policy and does not itself authorize action.

Because projection is defined as an explicit transformation of the state vector, its behavior remains inspectable and reproducible. Changes to the projection operator correspond to explicit revisions of the system's interpretive structure rather than hidden modifications to latent representations.

7 Policy Selection and Turn Outcomes

Once the system's current state has been expressed in policy-relevant coordinates and relevance gating has been evaluated, the Emergent State Machine determines a turn outcome.

If the current turn is not decision-relevant, then

$$g_t = 0 \;\Rightarrow\; r_t = \text{no-op}. \tag{15}$$

If the current turn is decision-relevant, deterministic policy is applied to the projected state:

$$a_t = \pi(y_t) \quad \text{when } g_t = 1 \tag{16}$$

where

- y_t is the policy-space representation of the current state,
- π is the policy selection function,
- a_t is the action selected by the system when action is authorized.

The resulting turn outcome is denoted by r_t. Possible outcomes include no-op, local action, escalation, handoff, or request for clarification. When an action is present, it is represented by a_t.

Policy rules encode the decision logic that governs system behavior. These rules may include threshold conditions, escalation logic, persistence requirements, cooldown constraints, routing logic, or explicit decisions to defer intervention under specified circumstances.

Because policy is deterministic, identical projected states will always produce identical policy results under fixed policy definitions. This property ensures that system decisions remain reproducible and traceable.

The full reasoning chain for a turn can therefore be expressed as

$$r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t. \tag{17}$$

When an action is present, the action itself is determined through

$$a_t \leftarrow \pi(y_t). \tag{18}$$

These chains allow any turn outcome to be traced backward through relevance gating, projection, state vector construction, signals, and observations.

In the Emergent State Machine architecture, policy represents the authoritative decision mechanism of the system. While projection interprets the state of the monitored system and gating determines whether the situation warrants policy evaluation, only policy determines which authoritative action may be taken in response.

8 State Evolution and Trajectories

As new observations arrive, the Emergent State Machine constructs new turns and updates its representation of the monitored situation. Each turn therefore produces a new state vector derived from the latest observations and any retained contextual structure.

Let o_{t+1} denote the observations available during the next turn. The system update can be written as

$$x_{t+1} = \Phi(x_t, o_{t+1}) \tag{19}$$

where Φ represents the deterministic transformation that incorporates new observations into the evolving state representation.

Successive state vectors form a trajectory through state space, a standard representation used in dynamical systems and control theory [Åström and Murray, 2010, Bertsekas, 1995]:

$$x_1, x_2, x_3, \ldots \tag{20}$$

Projection produces a corresponding trajectory through policy-relevant coordinates:

$$y_1, y_2, y_3, \ldots \tag{21}$$

This trajectory represents the temporal evolution of the monitored situation as it becomes relevant to decision.

The sequence of turns

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \ldots \tag{22}$$

therefore corresponds to a trajectory through the system's interpretable and projected state spaces.

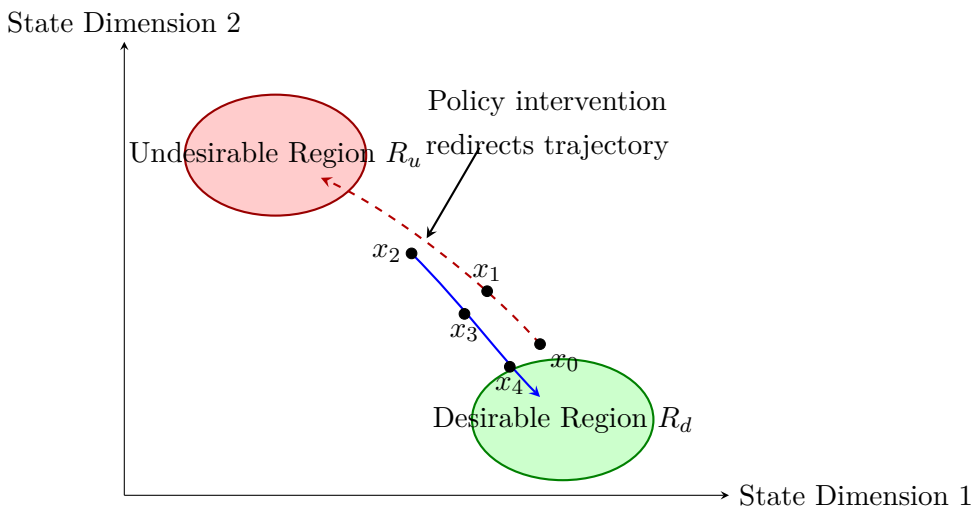

Figure 2: Example trajectory through ESM state space. Successive turns form trajectories that may converge toward desirable stable regions, persist within them, or diverge toward undesirable conditions.

Temporal relationships between observations may be incorporated into derived signals and projected dimensions. Such quantities may represent persistence, trends, rates of change, abrupt shifts, volatility over recent turns, or related measures of temporal structure. These temporal features allow the system to detect patterns that emerge over time rather than relying solely on instantaneous measurements.

Real-world time between turns may vary. Some turns may occur seconds apart, while others may be separated by minutes or longer intervals depending on the observation process and deployment context. The ESM therefore combines discrete reasoning order with variable elapsed time in the world being monitored.

Because temporal relevance is not uniform, the influence of some signals or derived conditions may decay over time. A disturbance, anomaly, or diagnostic concern may be highly relevant when recent and substantially less relevant after a sustained period of stability. The architecture therefore permits explicitly defined temporally decaying influence for signals or inferred conditions whose diagnostic relevance diminishes over time.

Trajectories through projected state space may converge toward desirable regions, persist within them, oscillate near their boundaries, or diverge away from them. Policy operates on this evolving representation to guide the system toward desirable conditions and away from undesirable ones.

9 Layered ESMs, Initialization, and Handoff

In many deployments, an Emergent State Machine does not operate in isolation. Multiple ESMs may operate at different levels of abstraction or over different aspects of the same monitored case.

A lower-level ESM may reason over a narrow local process, while a higher-level or earlier-phase ESM may construct broader contextual state. In such cases, downstream ESMs need not begin from context-free observations. Instead, they may begin from explicitly initialized state that has been produced by an upstream component or prior phase of reasoning.

This initialization may be represented abstractly as

$$x_1^{(d)} = H\Big(x_{\text{handoff}}^{(u)},\, o_1^{(d)}\Big) \tag{23}$$

where

- $x_{\text{handoff}}^{(u)}$ is the structured state provided by an upstream ESM or initializer,
- $o_1^{(d)}$ is the first local observation set of the downstream ESM,
- H is the deterministic handoff and initialization function,
- $x_1^{(d)}$ is the first downstream state vector.

This formulation clarifies two important architectural points.

First, initialization is invariant, but a distinct intake ESM is not. Every deployed ESM must begin from some initialized condition, but that initialization may be performed internally as part of the ESM's own first turn or externally through a distinct upstream process.

Second, layered ESMs interact through explicit state transfer and outcome-triggered coordination rather than through unrestricted continuous coupling. An upstream ESM may hand off structured state to downstream ESMs, but it need not remain a continuing live signal source after handoff. This preserves modularity, clear provenance, and replayability. This layered organization aligns with hierarchical control architectures in which higher-level systems establish context and constraints for lower-level processes without collapsing local autonomy [Albus et al., 1981].

Layered ESMs therefore allow shared situational context and specialized local reasoning to coexist without collapsing all interpretation and authority into a single undifferentiated system.

10 Desirable Regions and Stable States

Not all regions of projected state space correspond to acceptable or stable system conditions. Certain projected configurations represent preferred operational states, while others indicate

instability or risk. The Emergent State Machine therefore distinguishes between desirable and undesirable regions of projected state space.

Let $\mathcal{Y}$ denote the space of all possible projected state representations. Within this space, domain experts define a subset

$$\mathcal{Y}_D \subset \mathcal{Y} \tag{24}$$

representing desirable or stable projected system conditions.

For example, in a clinical monitoring system, the desirable region may correspond not merely to isolated physiological values but to projected conditions indicating sustained respiratory stability, acceptable uncertainty, and low recent volatility. In an educational system, desirable regions may correspond to projected combinations of reasoning coherence, evidence alignment, and sufficiently stable performance over recent turns.

As the system evolves through successive turns, the sequence of projected states

$$y_1, y_2, y_3, \ldots \tag{25}$$

forms a trajectory through projected state space. This trajectory may move toward or away from the desirable region $\mathcal{Y}_D$.

Operationally, desirable region membership is often temporal rather than instantaneous. A system may enter a desirable region briefly without yet being stable in any meaningful sense. Conversely, sustained residence within a desirable region over sufficient real-world time may justify stronger claims of stability, recovery, or readiness for transition. Persistence conditions therefore play a central role in determining when region membership has become decision-relevant.

A useful interpretation of attractor-like behavior in the ESM is therefore temporal and policy-relative. Desirable regions may function as stable attractors when trajectories tend to enter them, remain within them, and return to them after small perturbations under the combined influence of world dynamics and policy-guided intervention.

The ESM therefore provides a framework in which policy does not merely respond to isolated signals but instead shapes trajectories through projected state space over time.

11 Determinism, Transparency, and Auditability

A central design principle of the Emergent State Machine architecture is that its reasoning process remains deterministic and inspectable.

The transformation chain within each turn can be expressed as

$$o_t \to S_t \to x_t \to y_t \to g_t \to r_t \tag{26}$$

with policy evaluation contributing to the outcome whenever relevance gating activates authoritative decision.

Because each stage of this transformation is explicitly defined, system outcomes can be traced through the complete reasoning pathway. Any selected action can therefore be analyzed by examining the observations, signals, state vectors, projected dimensions, and gate conditions that produced the corresponding outcome.

This deterministic structure provides several important properties.

11.1 Replayability

Given recorded observations and fixed definitions of signals, projection, relevance gating, and policy, the system will reproduce the same turn outcome for a given turn. Interaction traces can therefore be replayed to reproduce previous system behavior.

Replayability enables retrospective analysis of system outcomes and allows designers to evaluate alternative projections, gating rules, or policies using historical interaction data.

11.2 Failure Localization

When undesirable outcomes occur, the source of the problem can be localized within the reasoning chain. Errors may originate from signal definitions, projection rules, relevance gates, temporal aggregation logic, or policy logic, but each stage remains independently inspectable.

Because each component of the architecture is explicit, debugging and system refinement can proceed through targeted revisions rather than opaque model retraining.

11.3 Auditability

The reasoning chain also supports formal auditing. For any turn outcome produced by the system, the full sequence of transformations from observation to projected state, relevance evaluation, and policy decision can be reconstructed.

This transparency is particularly important in domains where automated reasoning influences consequential outcomes, such as healthcare, education, or industrial operations.

Deterministic reasoning therefore provides a structural basis for system accountability.

12 Instrumented Deterministic Evolution

While the Emergent State Machine produces deterministic outcomes within a fixed configuration, real-world systems must evolve over time as domain knowledge improves and operational requirements change. The ESM therefore supports a regime of controlled architectural revision called *Instrumented Deterministic Evolution* (IDE).

Under IDE, system behavior evolves only through explicit revision of versioned architectural artifacts. These artifacts include

- signal definitions
- derived signal computations
- projection operators
- relevance-gating functions
- policy rules
- temporal aggregation or decay logic

Any modification to these components produces a new system configuration version.

Because each turn records the observations, signals, state vectors, projected states, gating results, and outcomes associated with a decision episode, historical interaction traces can be replayed under revised configurations. This enables designers to evaluate how policy revisions, projection changes, or gating changes would have affected past system behavior.

Formally, let v denote a specific configuration consisting of signal definitions, projection operators, gating logic, and policy rules. The outcome produced by the system during turn t can then be written abstractly as

$$r_t^{(v)} = R^{(v)}(o_t) \tag{27}$$

where $R^{(v)}$ denotes the full deterministic reasoning chain under configuration v.

When action is authorized under that configuration, the action may be written more specifically as

$$a_t^{(v)} = \pi^{(v)}\left(y_t^{(v)}\right) \qquad \text{when } g_t^{(v)} = 1. \tag{28}$$

Evaluating the same observations under a revised configuration v' yields a potentially different outcome

$$r_t^{(v')} = R^{(v')}(o_t). \tag{29}$$

Comparing these outcomes allows system designers to study how architectural revisions affect system behavior.

The key property of IDE is that behavioral change occurs only through explicit modification of versioned artifacts. System evolution therefore remains intentional and inspectable rather than emerging implicitly through hidden parameter drift.

Instrumented Deterministic Evolution allows systems to improve over time while preserving replayability, traceability, and governance over consequential decision logic.

13 Boundaries of the Architecture

The Emergent State Machine enforces structural separation between observation, interpretation, relevance evaluation, and authoritative policy selection. While this separation provides important properties of transparency and replayability, it does not eliminate the inherent limits of modeling, measurement, or governance.

13.1 Projection Quality

Projection determines how signals are interpreted as structured policy-relevant state. If signal definitions are incomplete, poorly designed, or biased, deterministic policy will faithfully act on those flawed representations.

The ESM therefore does not guarantee correctness of interpretation. Instead, it guarantees that interpretation remains explicit, inspectable, and revisable.

13.2 Policy Design

Policy encodes the authoritative decision logic of the system. Deterministic policy may still contain flawed thresholds, incorrect escalation logic, or undesirable governance assumptions.

The architecture ensures that such issues remain visible and modifiable. However, transparency alone does not guarantee that policies themselves are well designed.

13.3 Partial Observability and Clarification

Many real-world decision systems operate under partial observability, where the available signals may not uniquely determine the true system state. Formal frameworks such as partially observable Markov decision processes (POMDPs) model this uncertainty by maintaining belief distributions over latent states [Kaelbling et al., 1998].

The Emergent State Machine adopts a different architectural strategy. Rather than collapsing uncertainty through probabilistic belief updates within the decision mechanism, ambiguity is surfaced explicitly as a system condition. When available signals do not justify confident progress under the current reasoning constraints, the system may request clarification or additional observation before proceeding.

Clarification may arise either because relevance gating does not authorize consequential action under current conditions or because policy, once invoked, determines that additional information is required before authorizing action.

Ambiguity is therefore treated as an observable and governable system condition rather than an internal inference artifact.

13.4 Generative Assistance

Generative models may assist with tasks such as interpretation support, explanation generation, or interface construction. Within the ESM architecture, such models operate at the descriptive layer rather than the authoritative layer.

Generative outputs may contribute to observation processing, signal derivation, or projection support, but they cannot directly authorize actions. All consequential decisions must pass through deterministic relevance gating and policy evaluation.

This constraint allows generative capabilities to support system reasoning while preserving explicit governance over decision authority.

13.5 Action Boundary and Deployment Constraints

The ESM architecture does not imply unrestricted automation. In safety-critical deployments, the set of allowable automated actions may be externally constrained by institutional governance, protocol boundaries, or regulatory requirements.

Accordingly, action sets may be restricted to pre-authorized, explicitly bounded operations, while higher-risk or irreversible decisions remain reserved for human or supervisory authority. The architecture supports such constraints by ensuring that all authorized actions remain explicit, versioned, replayable, and externally governable.

13.6 Applicability Scope

The Emergent State Machine is most applicable in systems where automated reasoning influences externally consequential decisions. In domains where outputs are purely generative—such as creative text production—the strict separation between interpretation and authority may be unnecessary.

The architecture therefore represents a design discipline for systems in which interpretability, accountability, and governance of automated decisions are critical requirements.

14 Related Work

The Emergent State Machine (ESM) draws on ideas from several established traditions, including control theory, dynamical systems, software architecture, situation awareness research, and recent work on AI interpretability and governance. While each of these areas contributes elements relevant to the architecture, the ESM combines them in a distinct way by enforcing a structural separation between measurement, interpretation, relevance evaluation, and authoritative policy selection.

14.1 Control and State-Space Systems

Control theory has long represented system behavior using explicit state vectors and state-space models [Kalman, 1960, Åström and Murray, 2010]. Classical formulations distinguish between measurements, state estimation, and control input, while decision-theoretic control frameworks treat action selection as an explicit policy over system state [Bertsekas, 2017]. Observers estimate system state from measurements, while controllers determine actions based on the estimated state.

The ESM shares this structural intuition. Signals correspond to measured quantities, the state vector provides an explicit representation of the system's condition, projection expresses that condition in policy-relevant coordinates, and policy governs authoritative action selection. However, modern decision systems—particularly those incorporating machine learning—often collapse estimation and control into a single opaque model. The ESM reinstates explicit architectural boundaries among descriptive state representation, relevance determination, and authoritative policy selection.

14.2 Dynamical Systems and Trajectories

The representation of system evolution as trajectories through state space is central to dynamical systems theory. Concepts such as attractors, stability regions, and phase-space trajectories provide a mathematical framework for understanding how systems evolve over time and how interventions influence those trajectories [Strogatz, 2015].

The ESM adopts this perspective by treating successive turns as trajectories through interpretable and projected state space. Policy-guided interventions act to steer these trajectories toward desirable regions and away from unstable configurations.

14.3 Situation Awareness

Research in human factors has emphasized the importance of situation awareness in complex operational environments. Situation awareness describes the process by which operators perceive relevant signals, interpret their meaning, and project the implications for system behavior [Endsley, 1995].

The ESM architecture provides a computational analogue to this process. Observations become signals, signals are assembled into a structured state representation, projection expresses that state in policy-relevant terms, and relevance-gated policy evaluates whether and how the system should respond. In this sense, the architecture can be viewed as a formalization of situational reasoning within a deterministic computational framework.

14.4 Software Architecture and Separation of Concerns

The idea that complex systems benefit from explicit architectural separation has deep roots in systems science and software engineering. Simon's analysis of hierarchical systems showed that complex systems become manageable when organized into modular structures with clearly defined responsibilities [Simon, 1962]. Later work in software architecture extended these ideas through principles such as modular decomposition and separation of concerns [Parnas, 1972, Dijkstra, 1974]. Architectural patterns such as Model–View–Controller further demonstrate how separating representation from control logic can reduce coupling between system components [Reenskaug, 1979].

The ESM extends this architectural philosophy to automated reasoning systems. Measurement, interpretation, relevance evaluation, and decision authority are separated into explicit stages, preventing probabilistic interpretation mechanisms from directly authorizing consequential actions.

14.5 AI Interpretability and Governance

Recent work in AI interpretability and governance has highlighted the risks associated with opaque decision systems [Doshi-Velez and Kim, 2017, Mittelstadt et al., 2016]. Many modern architectures embed both representation and decision logic within learned model parameters, making it difficult to trace decisions to specific inputs or policy assumptions.

The ESM addresses this challenge by ensuring that the reasoning chain from observation to outcome remains explicit. Signals, projections, gates, and policies are defined as inspectable artifacts whose behavior can be examined and revised independently. This structural transparency enables replayability, failure localization, and controlled system evolution under Instrumented Deterministic Evolution.

14.6 Distinguishing Features of the ESM

While each of the above traditions contributes ideas relevant to the architecture, the Emergent State Machine introduces a distinctive combination of principles:

- reasoning is organized into explicit **turns**
- system state is represented through **interpretable signals and state vectors**
- projection expresses state in **policy-relevant coordinates**
- relevance gating determines when **authoritative decision is warranted**
- authoritative outcome selection remains **deterministic and inspectable**
- layered deployments preserve **modular state transfer and local authority**

- system evolution occurs through **Instrumented Deterministic Evolution**

Together, these elements produce an architecture in which automated reasoning remains transparent, reproducible, and governable even as systems evolve over time.

15 Conclusion

This paper has presented the mathematical foundations of the Emergent State Machine (ESM), a deterministic architecture for interpretable situational reasoning in complex systems. The architecture organizes reasoning into discrete units called turns, each of which transforms observations into signals, constructs an explicit state vector, projects that state into policy-relevant coordinates, evaluates relevance, and, when warranted, deterministically produces an action or other outcome.

By structuring system behavior as a sequence of turns, the ESM makes reasoning processes inspectable, replayable, and auditable. State evolution becomes visible as trajectories through interpretable and projected state space, allowing policies to guide the system toward desirable regions and away from unstable conditions over variable real-world time.

The architecture preserves structural boundaries among interpretation, relevance evaluation, and authority. Projection mechanisms describe the current situation, relevance gating determines whether the situation warrants policy evaluation, and deterministic policy governs consequential action. This separation ensures that system behavior remains traceable to explicit architectural artifacts rather than hidden model parameters.

Because signals, projections, gates, temporal logic, and policies are explicitly defined and versioned, the architecture supports a regime of Instrumented Deterministic Evolution in which behavioral change occurs only through intentional revision of system components. System evolution therefore becomes controlled and inspectable rather than implicit.

Layered deployments extend these properties by allowing initialized state to be transferred across interacting ESMs without collapsing modularity or obscuring provenance. The result is a framework in which local reasoning, shared context, temporal persistence, and deterministic authority can coexist within a single inspectable architecture.

While the architecture does not eliminate the challenges of modeling complex domains, it provides a disciplined framework in which automated reasoning remains transparent, reproducible, and governable. In systems where probabilistic interpretation influences consequential decisions, maintaining explicit boundaries among description, relevance, and authority is not merely a design preference but an architectural requirement.

References

James S. Albus, Tony Barbera, and R. Nagel. Theory and practice of hierarchical control. In *Proceedings of the 23rd IEEE Computer Society International Conference*, Washington, DC, USA, 1981.

Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010.

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, 1995.

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control, Vol. I*. Athena Scientific, 2017.

Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

Edsger W. Dijkstra. On the role of scientific thought. *Selected Writings on Computing: A Personal Perspective*, pages 60–66, 1974.

Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

Mica R. Endsley. Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1):32–64, 1995.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. *Planning and Acting in Partially Observable Stochastic Domains*, volume 101. 1998.

Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82:35–45, 1960.

Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 1999.

Brent Mittelstadt, Patrick Allo, Mariarosaria Taddeo, Sandra Wachter, and Luciano Floridi. The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2), 2016.

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

Srinivasa Raghavan and Sanjeev Kulkarni. *Signal Detection and Estimation*. Springer, 2014.

P. J. Ramadge and W. M. Wonham. Supervisory control of a class of discrete event processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. doi: 10.1137/0325013.

Trygve Reenskaug. Models–views–controllers. *Xerox PARC Technical Note*, 1979.

Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

Steven H. Strogatz. *Nonlinear Dynamics and Chaos*. Westview Press, 2015.

Paulo Tabuada. Event-triggered real-time scheduling of stabilizing control tasks. *IEEE Transactions on Automatic Control*, 52(9):1680–1685, 2007.